

GDA Stability and Lockdown Standard

Purpose

The purpose of this standard is to stop working features from regressing after they are fixed. The platform already has a documented testing and response-contract foundation; this standard turns those ideas into release rules.[cite:216] [cite:218]

Lockdown principles

1. One source of truth

The React source must live in one controlled source-of-truth repository or equivalent controlled source flow. Future updates cannot depend on ad hoc copies, manual zip bundles, or ambiguous live-server edits.[cite:215]

2. One production surface

React is the production UI. Retool should be archived/reference-only so product work is not split across parallel interfaces.[cite:215]

3. One response contract

Every React-facing webhook or endpoint must return the standard JSON envelope with `success`, `workflow`, `action`, `dryRun`, `data`, `meta`, and `error`. React must not be forced to guess what an empty body, HTML error page, or blank string means.[cite:216]

4. One deploy path

There should be one canonical production deploy path. Alternate or experimental deploy methods must be archived, documented as non-production, or removed once replacement is live.[cite:218] [cite:251]

Release gate

No production release should be considered valid unless the following pass:

- Smoke checks
- API proxy checks
- Workflow smoke for the affected areas
- Read-only E2E journeys for the affected user paths
- Dry-run E2E for any write-capable path if relevant

The existing E2E test map already defines these levels and safety categories.[cite:218]

Required test classes

Smoke

Confirms pages load, no major client errors appear, and critical routes respond.[cite:218]

API contract

Confirms `/api/...` endpoints return the standard JSON envelope and use structured success/failure semantics.[cite:216]

Business-journey E2E

Confirms real user journeys work through React, API, workflow, data layer, and back to UI. These tests are higher value than simple route checks because they prove business behavior rather than technical availability alone.[cite:218]

Data integrity

Confirms any writes are correct, test-safe, reversible, dry-run capable, or approval-gated as required.[cite:218]

Safety lanes

Every workflow should be classified into one of the documented safety lanes:

- Read-only
- Dry-run
- Test-row
- Approval required
- Unknown

Unknown workflows should not be automated or wired into high-trust product surfaces until classified.[cite:215]

Definition of locked feature

A feature is considered locked only when all of the following are true:

- It has a stable source-controlled implementation.[cite:215]
- Its endpoints follow the standard response contract.[cite:216]
- It has smoke coverage.
- It has at least one relevant business-journey E2E check where appropriate.[cite:218]
- It is part of the canonical deploy path.
- Its failures are visible through QA/health reporting rather than hidden behind blank states.[cite:215][cite:218]

Change-control policy

Changes to locked features must go through:

1. Planned scope definition
2. Contract-aware implementation
3. Test execution
4. Release verification
5. Post-release health check

If a change cannot satisfy these steps, it should not be applied directly to the production feature.

Immediate implementation targets

The first areas to bring under this standard are:

- Launchpad
- Ops Tracker
- Pipeline separation
- Persistent financial KPI header
- Platform health / QA reporting

These are the highest-leverage surfaces for rebuilding trust in the platform.[cite:215][cite:218]